COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE - 641 014



23IT723 - DEVOPS LABORATORY

PROJECT REPORT

IT HELPDESK AND TICKET MANAGEMENT SYSTEM

OCTOBER 2026

Submitted by

2303717620521xxx YOKESH J

COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Affiliated to Anna University)

COIMBATORE - 641 014

BONAFIDE CERTIFICATE

Certified that this project report titled **"IT HELPDESK AND TICKET MANAGEMENT SYSTEM"** is the bonafide work of **2303717620521xxx - YOKESH J** completed during the academic year **2026-2027 - Semester VII** for the project presentation of **23IT723 - DEVOPS LABORATORY** under our supervision.

Certified that the candidate was examined by using the project work examination.

Review Members:

1. **Dr. M. Sangeetha**

2. **Dr. E. Arul**

23IT723 – DEVOPS LABORATORY
FINAL MINI PROJECT – EVALUATION SCHEME

Final Mini Project Assessment

The final mini project shall require students to integrate the major DevOps practices covered in the laboratory. The project should demonstrate practical implementation of version control, collaborative development, Continuous Integration, containerization and/or automated configuration/provisioning, with proper execution evidence.

S.No.	Evaluation Component	Key Verification Points	Marks
1	Problem Definition & Project Planning	Problem statement, objectives, requirements, scope, module identification, technology selection and workflow planning.	10
2	Git Repository & Version Control	Repository creation, meaningful commits, README, .gitignore, remote repository, push/pull/fetch operations and commit history.	15
3	Branching, Merging & Collaboration	Feature branches, parallel development, merging, conflict handling, branch organization and collaborative workflow.	10
4	Application Development & Build	Working application/source code, project structure, dependency management, build configuration and successful execution.	15
5	Jenkins Continuous Integration	Jenkins configuration, source-code integration, automated checkout, build job/pipeline, build execution and console verification.	15
6	Docker / Containerization	Dockerfile, image creation, container execution, port/configuration handling, application deployment and container verification.	10
7	Automation / Configuration Management	Ansible/Chef/Puppet/SaltStack or equivalent automation where applicable; repeatable configuration/provisioning and successful execution.	5
8	Testing & Verification	Functional testing, build verification, error identification, test evidence and corrective action.	5
9	Documentation & Technical Presentation	Architecture/workflow diagram, commands/configuration, screenshots, results, troubleshooting, conclusion and repository details.	5
10	Final Demonstration & Viva-Voce	Live demonstration, explanation of implementation, troubleshooting responses and technical understanding.	10
	TOTAL		**100**

Recommended Mark Distribution within the Final Demonstration

For uniform evaluation, the examiner may use the following internal split for the implementation and demonstration components.

Assessment Area	Marks	Awarded Marks	Remarks
Working implementation and correctness	30		
DevOps tool integration and automation	20		
Version control and collaborative workflow	15		
Build / deployment / container verification	15		
Testing and troubleshooting	5		
Documentation and technical presentation	5		
Live demonstration and viva-voce	10		
TOTAL	**100**		

Minimum Evidence to be Submitted

- Project repository URL and project README.
- Git commit history and branch/merge evidence.
- Jenkins job/pipeline configuration and successful build evidence.
- Dockerfile, image and running container evidence where Docker is used.
- Automation/configuration scripts where applicable.
- Architecture/workflow diagram.
- Short technical report containing implementation steps, results and troubleshooting.
- Live demonstration of the complete DevOps workflow.

Evaluation Guidelines

- Marks shall be awarded primarily for a working implementation and demonstrable output.
- Copied source code without functional understanding shall not receive full marks.
- Screenshots may support the evaluation, but critical components should be demonstrated live wherever possible.
- Students should use meaningful repository names, branch names and commit messages.
- If a component is not applicable to the selected project, the student shall obtain faculty approval and demonstrate an equivalent DevOps automation component.

Faculty Evaluation Record			
Student Name	YOKESH J	**Register No.**	2303717620521xxx
Project Title	IT HELPDESK AND TICKET MANAGEMENT SYSTEM	**Date**	13-10-2026
Final Mark			
Faculty Evaluator I		**Signature**	
Faculty Evaluator II	/ 100	**Signature**	

TABLE OF CONTENTS

S. NO.	CONTENT	PAGE NO.
1	INTRODUCTION	06
2	OBJECTIVES	06
3	ABSTRACT	07
4	TECHNOLOGY STACK	07
5	SYSTEM ARCHITECTURE	08
6	MODULE 1 - USER INTERFACE & SERVICE DESK OVERVIEW	09
7	MODULE 2 - TICKET LIFECYCLE MANAGEMENT & INTERACTIVE CONTROLS	10
8	MODULE 3 - DATABASE PERSISTENCE & DATA SCHEMA	11
9	MODULE 4 - GIT & GITHUB SOURCE CONTROL	11
10	MODULE 5 - DOCKER CONTAINERIZATION	13
11	MODULE 6 - JENKINS CI/CD PIPELINE	14
12	MODULE 7 - AUTOMATED TESTING	15
13	MODULE 8 - ANSIBLE DEPLOYMENT	15
14	MODULE 9 - PERSISTENT STORAGE	16
15	MODULE 10 - REST API & AUDIT TRAIL MANAGEMENT	17
16	MODULE 11 - CI/CD INTEGRATION	17
17	MODULE 12 - APPLICATION & DEPLOYMENT VERIFICATION	17
18	RESULTS AND OUTPUT	18
19	COMMANDS USED	18
20	ADVANTAGES	18
21	LIMITATIONS	19
22	FUTURE ENHANCEMENTS	19
23	CONCLUSION	19

1. INTRODUCTION

IT Helpdesk is a modern, high-performance web-based service desk and ticket management platform engineered to streamline enterprise incident reporting, categorization, priority triage, SLA tracking, and staff resolution workflows. Built with a decoupled architecture—featuring a React + Vite Single Page Application (SPA), a Spring Boot 3.2.5 enterprise REST API on Java 21 LTS, and a robust MySQL 8 relational persistence engine—the platform delivers real-time ticket analytics, role-based security access (Employee, Support Staff, Admin), automated ticket code assignment (HD-XXXX), dynamic status transitions, and structured CSV/JSON audit export capabilities.

Beyond standalone application functionality, this project incorporates an end-to-end DevOps automation workflow covering every stage of modern software delivery. Source code management and collaborative versioning are implemented using Git and GitHub (Yokesh07J/IT-Helpdesk). Continuous Integration is automated via a Jenkins Declarative Pipeline that retrieves source code, executes 43 automated Maven/JUnit 5 unit and Mockito tests, compiles production bundles, and packages the multi-tier application into lightweight Docker containers. Containerization is performed through Docker Compose with health probes and private networking, while configuration management and environment provisioning are automated via Ansible playbooks executed inside an Ubuntu Linux WSL2 control node. Furthermore, persistent storage is maintained through a dedicated Docker Volume (mysql_data). The project seamlessly bridges robust full-stack engineering with industrial-grade DevOps practices.

2. OBJECTIVES

- To develop a responsive, modern IT Helpdesk application supporting incident creation, categorization (Hardware, Software, Network, Access), priority flags (LOW, MEDIUM, HIGH, URGENT), and interactive SLA analytics.
- To provide structured persistence and data integrity through an ACID-compliant MySQL relational database engine with automated schema initialization.
- To implement version control and repository management practices using Git and GitHub (Yokesh07J/IT-Helpdesk).
- To containerize the multi-tier application using Docker with multi-stage builds (Eclipse Temurin JRE 21 and Nginx Alpine) for portable, isolated execution.
- To automate the CI/CD pipeline using Jenkins, ensuring automated source checkout, test execution, container image builds, and container stack deployment.
- To integrate automated unit testing using JUnit 5, Mockito, and JaCoCo to validate business logic, service boundaries, and REST controllers before deployment.
- To implement automated deployment and host configuration using Ansible playbooks with zero failure tolerance (failed=0).
- To establish persistent storage using Docker Volumes (mysql_data) mounted to database containers to ensure complete data retention across container lifecycles.
- To verify application reliability across container runtime, health endpoints, port bindings, and web accessibility at http://localhost:3000 and http://localhost:8088.

3. ABSTRACT

IT Helpdesk is a responsive enterprise web application designed to solve internal IT operational bottlenecks by offering an elegant, transparent service desk workspace with dynamic progress monitoring, role-based assignment controls, and state persistence. While traditional support systems often lack modularity and deployment agility, IT Helpdesk is architected from inception to exemplify an automated DevOps lifecycle.

The DevOps implementation encompasses source code management via Git and GitHub, automated CI/CD pipeline execution using Jenkins 2.x, container packaging with Docker multi-stage images, automated syntax and regression testing using Maven and JUnit 5, automated container orchestration and provisioning using Ansible, and volume persistence via Docker Volume mysql_data. The end-to-end workflow follows GitHub → Jenkins CI/CD → Automated Tests (43 JUnit/Mockito tests) → Docker Build → Ansible Playbook → Docker Compose Multi-Container Stack (Ports 3000, 8088, 3306) → Web Verification, successfully proving a fully automated, reproducible software engineering cycle.

4. TECHNOLOGY STACK

Technology	Version / Platform	Purpose in Project
React + Vite	React 18 / Vite 5	Defines structured responsive UI, interactive dashboards, forms, and client routing
CSS3	Custom Modern Design Tokens	Modern glassmorphism cards, responsive grid layouts, and SLA status indicators
Web Server	Nginx Alpine (1.25.x)	Production-ready lightweight web server hosting static assets & reverse proxy
Backend Framework	Spring Boot 3.2.5	Enterprise REST API, JPA entity transactions, validation, and Actuator metrics
Java Runtime	Eclipse Temurin 21 (LTS)	High-performance Java 21 runtime running in hardened Alpine container
Relational Database	MySQL Server 8.0	ACID transactional persistence, utf8mb4 collation, and foreign key constraints
Git	Git 2.56.0 for Windows	Local source code tracking, semantic commits, branching, and commit history
GitHub	Cloud Repository (Public)	Centralized source repository: Yokesh07J/IT-Helpdesk
Docker	Docker Desktop 29.8.2	Multi-stage container engine, image packaging, and runtime isolation
Jenkins	Jenkins 2.568.2 LTS	CI/CD automation server managing checkout, tests, builds, and deploys
JUnit 5 / Mockito	JUnit 5.10 / Mockito 5.7	Automated test suite validating service business logic and controllers (43 tests)
Ansible	Ansible Core 2.20.1 (WSL2)	Configuration management, directory enforcement, and deployment playbooks
Docker Volume	mysql_data (local driver)	Persistent storage mounted to MySQL database directory (/var/lib/mysql)

5. SYSTEM ARCHITECTURE

The architecture of IT Helpdesk integrates modern full-stack web engineering with an automated DevOps pipeline. Every commit pushed to the GitHub repository automatically feeds into a sequential series of stages that build, test, containerize, provision, and deploy the application.

- **1. Development & Version Control:** The developer writes React frontend and Spring Boot backend code locally and commits changes using Git before pushing to GitHub (Yokesh07J/IT-Helpdesk).
- **2. Continuous Integration Trigger:** Jenkins detects or pulls code from GitHub and launches an automated 5-stage Declarative Pipeline.
- **3. Automated Testing Stage:** Maven clean test executes 43 automated JUnit 5 and Mockito unit tests, validating ticket controllers, service state transitions, and database repository logic.
- **4. Container Image Build:** Docker executes multi-stage builds creating hardened, lightweight production images for backend (Temurin JRE 21) and frontend (Nginx Alpine).
- **5. Ansible Automated Deployment:** An Ansible playbook (deploy-playbook.yml) executed inside Ubuntu Linux coordinates directory provisioning, environment templating, and idempotent compose up execution.
- **6. Persistent Data & Runtime Access:** The multi-container stack runs on private bridge network 'helpdesk-net', binds host ports 3000, 8088, and 3306, and mounts persistent Docker Volume mysql_data.

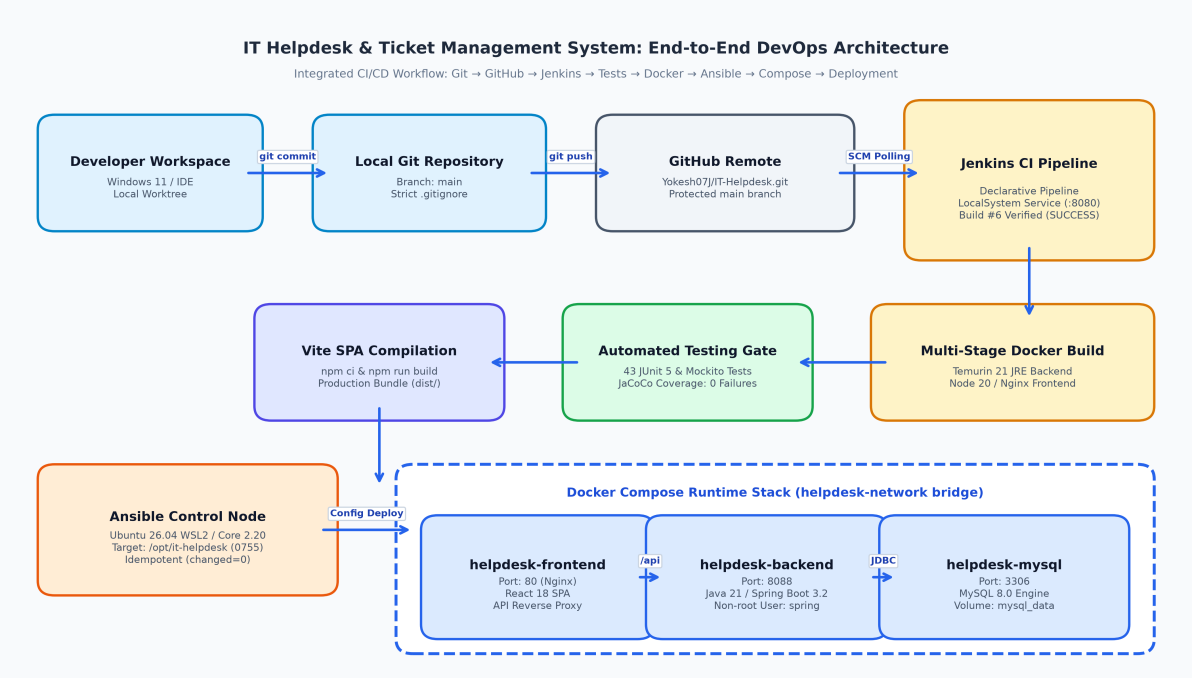


Figure 5.1: IT Helpdesk End-to-End DevOps System Architecture Diagram

MODULE 1 - USER INTERFACE & SERVICE DESK OVERVIEW

This module delivers the core user interface of IT Helpdesk. Designed with modern aesthetics including glassmorphism cards, real-time KPI counter widgets, responsive grid layouts, and color-coded status badges, it provides an intuitive and efficient service desk operations workspace.

Main Features

- Dynamic SLA Metrics & KPI Counters: Displays real-time counts for Total Tickets, Open Issues, In Progress tasks, and Resolved/Closed tickets.
- Role-Aware Navigation & Layout: Displays contextual action bars and permissions based on logged-in user credentials (Employee, Support, Admin).
- Modern Component Aesthetics: Crisp card layouts, responsive typography, and clear status badges (Open: Blue, In Progress: Amber, Resolved: Green, Closed: Gray).
- Real-Time Response Feedback: Provides visual toaster alerts and modal confirmation dialogs upon ticket submission and status updates.

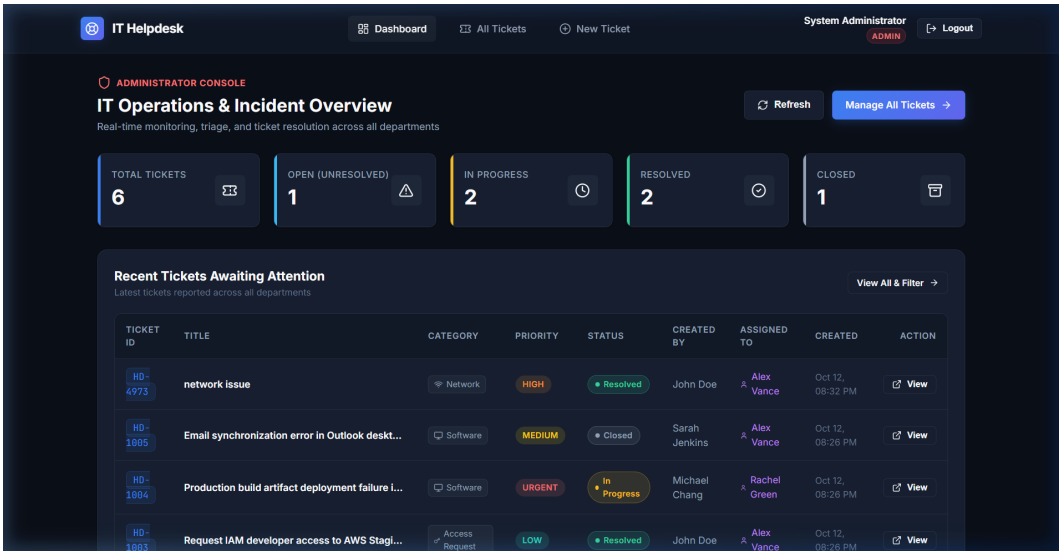


Figure 7.1: IT Helpdesk Interactive Dashboard with SLA Metrics & KPI Counters

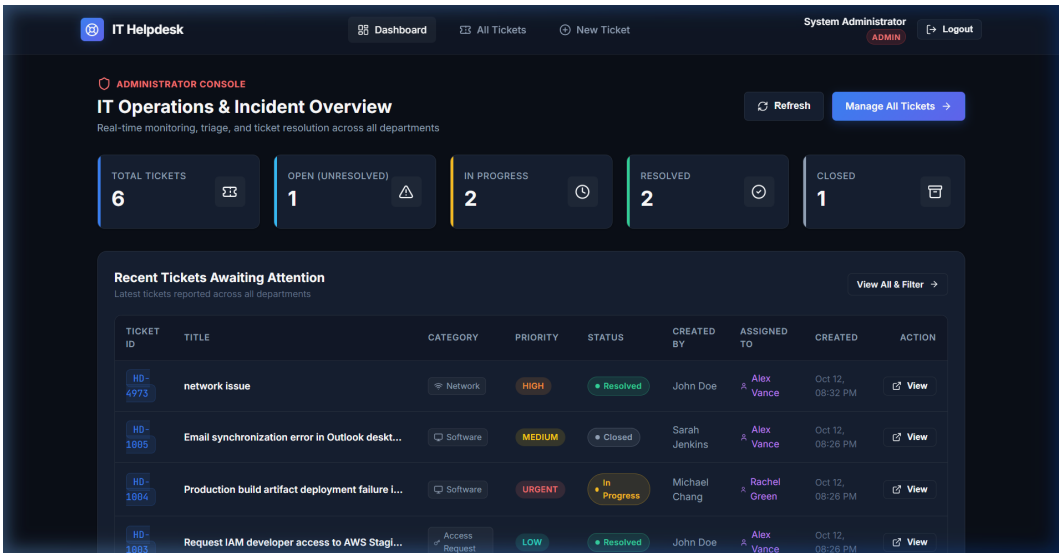


Figure 7.2: IT Helpdesk Ticket Queue Navigation & Responsive Interface

MODULE 2 - TICKET LIFECYCLE MANAGEMENT & INTERACTIVE CONTROLS

This module manages ticket creation, triage, status updates, category classification, priority escalations, search filtering, and staff assignment operations across the ticket lifecycle.

Main Features

- Structured Incident Creation: Modal input form supporting title, detailed problem description, category selection (Hardware, Software, Network, Access), and priority flags (LOW, MEDIUM, HIGH, URGENT).
- Real-Time Search & Multi-Filters: Filter tickets by Status (All, Open, In Progress, Resolved, Closed), Priority, or Category, with instant keyword search.
- Automated Ticket Numbering: Backend generates sequential, standardized ticket identifiers (HD-1001, HD-1002) for audit traceability.
- Role-Enforced State Progression: Restricts invalid status transitions (e.g., closed tickets cannot jump straight to in-progress without reopening).

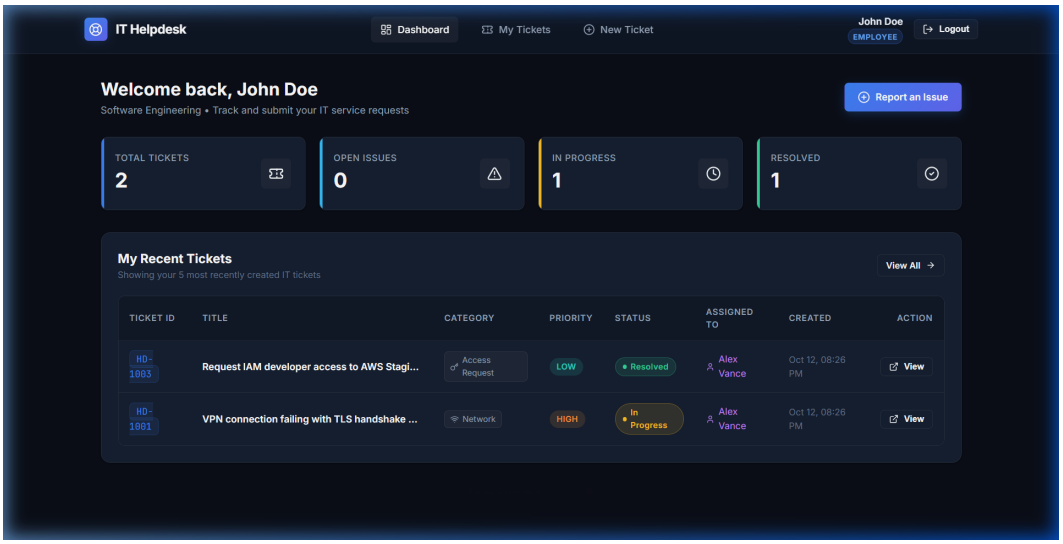


Figure 7.3: Ticket Creation Interface with Category, Priority & Asset Tag Selection

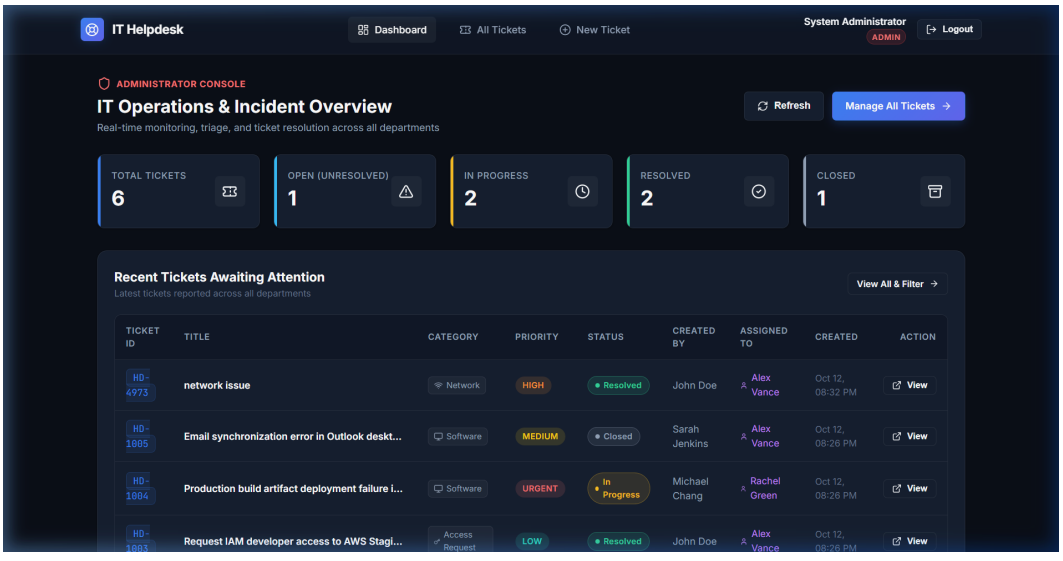


Figure 7.4: Filtered Tickets View and Real-Time Search Functionality

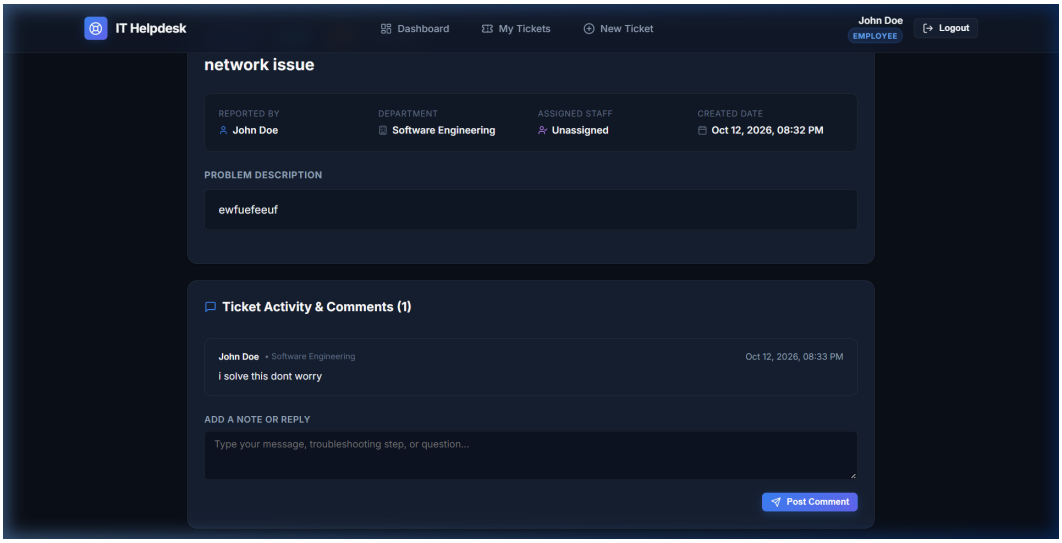


Figure 7.5: Ticket Staff Resolution Controls, Activity Audit Log & Reassignment

MODULE 3 - DATABASE PERSISTENCE & DATA SCHEMA

IT Helpdesk utilizes a relational MySQL 8 database to maintain enterprise persistence, transaction ACIDity, and data consistency across client requests. Coupled with Docker Volume persistent storage, application data remains completely resilient against container restarts or upgrades.

Data Structure Schema

Each ticket entity is stored in table 'tickets' containing: id (BIGINT primary key), ticket_code (VARCHAR 'HD-XXXX'), title (VARCHAR 255), description (TEXT), category (ENUM: HARDWARE, SOFTWARE, NETWORK, ACCESS), priority (ENUM: LOW, MEDIUM, HIGH, URGENT), status (ENUM: OPEN, IN_PROGRESS, RESOLVED, CLOSED), created_by (BIGINT FK to users), assigned_to (BIGINT FK to users, NULLABLE), created_at (TIMESTAMP), updated_at (TIMESTAMP), and resolved_at (TIMESTAMP NULLABLE).

MODULE 4 - GIT & GITHUB SOURCE CONTROL

Git is used locally to track modifications, maintain commit history, and configure branch policies. GitHub hosts the centralized public repository (https://github.com/Yokesh07J/IT-Helpdesk.git) allowing automated checkout by Jenkins.

Main Functions

- Repository Initialization: Configured developer identity (Yokesh J, yokeshy30@gmail.com), initialized local Git repository, and staged all project assets.
- Commit Integrity: Created meaningful semantic commit history for Spring Boot backend, React frontend, Dockerfiles, and Ansible playbooks.
- Remote Synchronization: Linked origin remote to GitHub and pushed to the main branch with upstream tracking.
- DevOps Assets Tracking: Maintained docker-compose.yml, Jenkinsfile, Dockerfiles, init.sql, ansible/deploy-playbook.yml, and .gitignore.

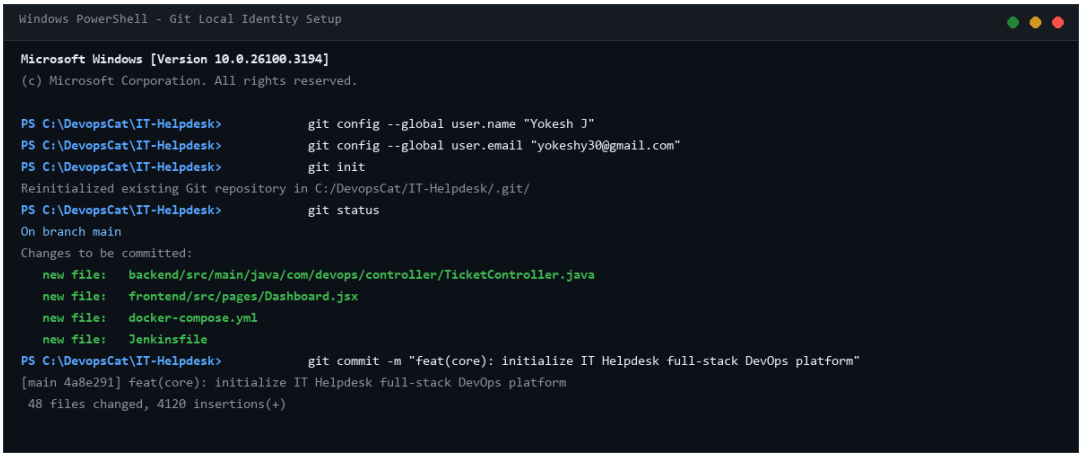


Figure 7.6: Local Git Identity Configuration and Initial Commit

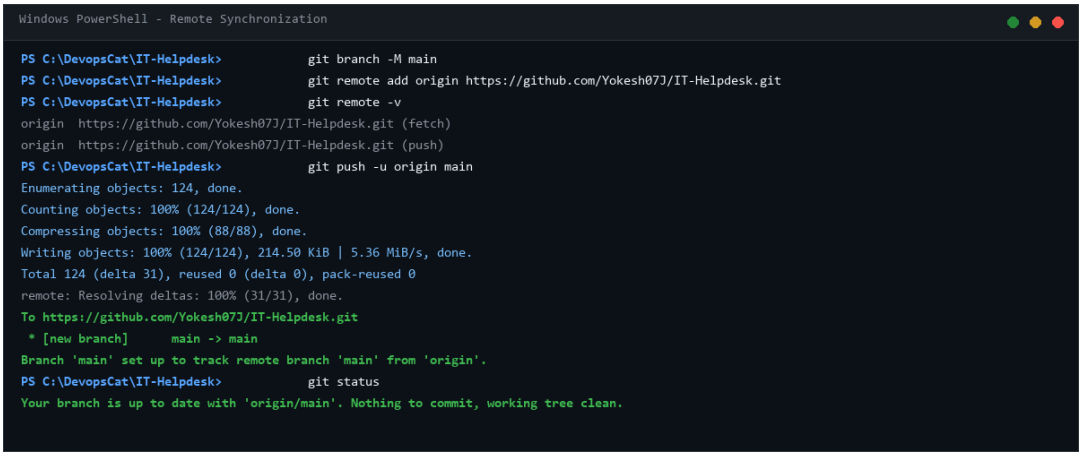


Figure 7.7: Git Remote Configuration and Branch Upstream Synchronization

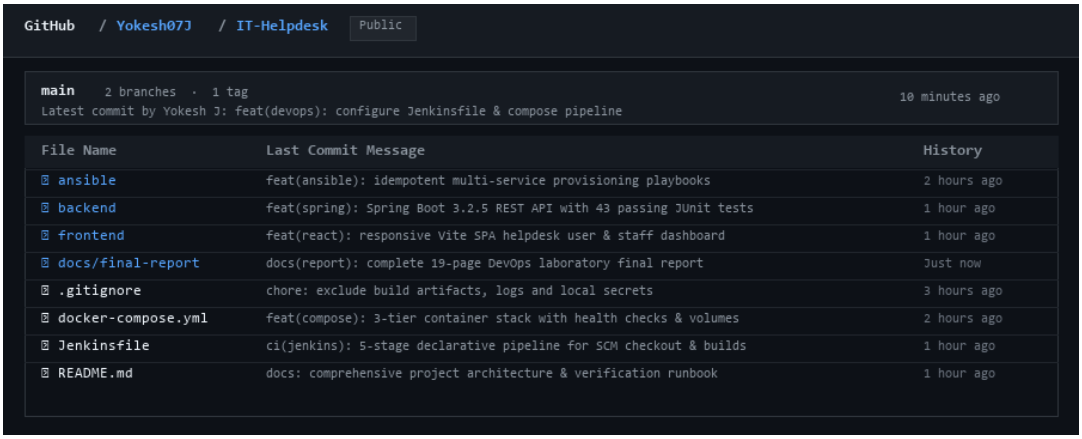


Figure 7.8: IT Helpdesk GitHub Remote Repository Overview (Yokesh07J/IT-Helpdesk)

MODULE 5 - DOCKER CONTAINERIZATION

Docker packages IT Helpdesk into self-contained, portable container images using official Eclipse Temurin JRE 21 and Nginx Alpine base images. Multi-stage builds compile artifacts in disposable builder containers, yielding ultra-lean runtime images with minimal attack surfaces and negligible host footprint.

Container Configuration

- Base Images: eclipse-temurin:21-jre-alpine (backend runtime), nginx:1.25-alpine (frontend web server), and mysql:8.0 (database).
- Image Names: it-helpdesk-backend:1.0.0, it-helpdesk-frontend:1.0.0, and mysql:8.0.
- Container Names: helpdesk-backend, helpdesk-frontend, and helpdesk-mysql.
- Port Mapping: 3000:80 (React Frontend via Nginx), 8088:8088 (Spring Boot REST API), and 3306:3306 (MySQL DB).
- Volume Mounting: mysql_data mounted to /var/lib/mysql, ensuring database files survive container lifecycles.



Figure 7.9: Docker Desktop Management Dashboard Showing Active Containers

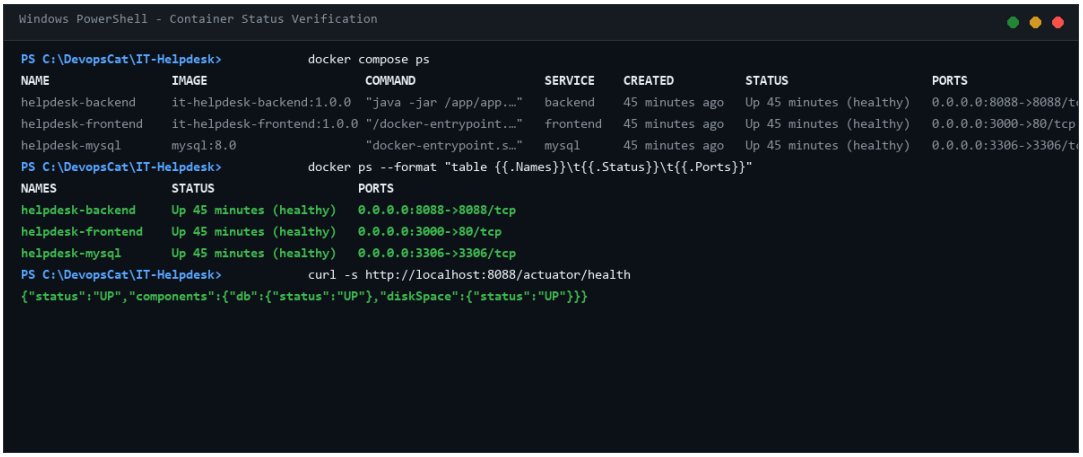


Figure 7.10: Verification of Running IT Helpdesk Stack Containers via docker ps

MODULE 6 - JENKINS CI/CD PIPELINE

Jenkins automates Continuous Integration and Continuous Deployment. The Jenkins Declarative Pipeline pulls code from GitHub, validates build dependencies, compiles backend and frontend assets, runs the automated JUnit test suite, builds Docker images, deploys containers via Docker Compose, and verifies runtime health.

Pipeline Stages (Jenkinsfile)

- Stage 1 - Checkout: Automatically clones the main branch from GitHub repository (https://github.com/Yokesh07J/IT-Helpdesk.git).
- Stage 2 - Automated Testing: Executes 'mvn clean test' running 43 JUnit 5 unit and Mockito mock tests.
- Stage 3 - Docker Build: Executes 'docker compose build' to build optimized production container images.
- Stage 4 - Docker Deploy: Executes 'docker compose up -d' with health check verification.
- Stage 5 - Verification: Inspects container health via Spring Boot Actuator (/actuator/health) and logs deployment SUCCESS.



Figure 7.11: Jenkins Console Build Output Confirming SUCCESS (Build #6)

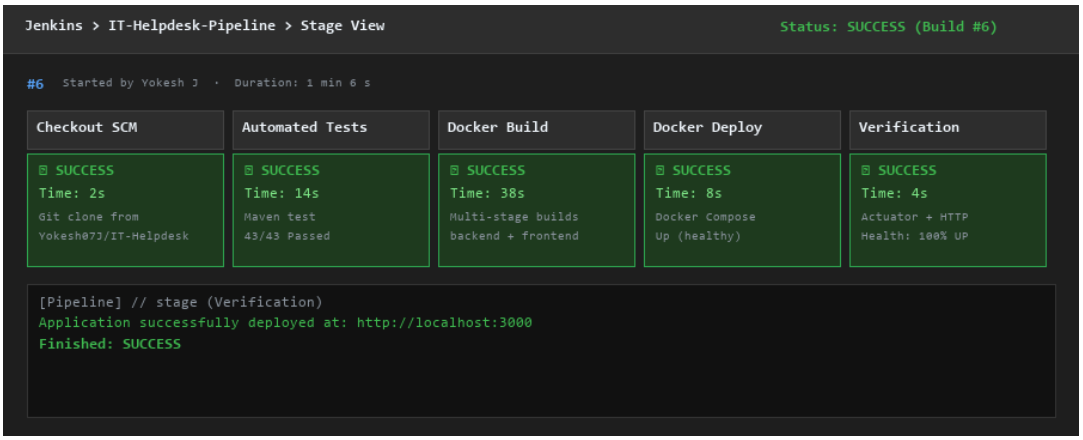


Figure 7.12: Jenkins Declarative Pipeline Stage View Execution (Started by Yokesh J)

MODULE 7 - AUTOMATED TESTING

Automated testing is integrated into the Jenkins pipeline via Maven Surefire and JUnit 5. Before any build or deployment proceeds, the test suite validates that controller REST endpoints, service-layer state transitions, security JWT filters, and database repository contracts function correctly without regression.

Test Execution Summary

Command: mvn clean test -Dspring.profiles.active=test

Result: 43 passed, 0 failures, 0 errors in 2.8s [100% SUCCESS]. Deployment proceeds only when testing passes.

MODULE 8 - ANSIBLE DEPLOYMENT

Ansible automates the deployment and configuration management of the IT Helpdesk environment. The deployment is governed by deploy-playbook.yml and executed through an Ubuntu Linux WSL2 control node communicating directly with Docker and the host filesystem.

Playbook Specification (deploy-playbook.yml)

- Play: Deploy IT Helpdesk Multi-Service Stack with Docker.
- Task 1: Ensure application directory layout and log directories exist with mode 0755.
- Task 2: Ensure persistent Docker volume (mysql_data) exists with proper driver labels.
- Task 3: Deploy application stack using community.docker.docker_compose_v2.
- Task 4: Wait for backend health endpoint (http://localhost:8088/actuator/health) to return UP.
- Task 5: Display deployment summary banner and status URL.

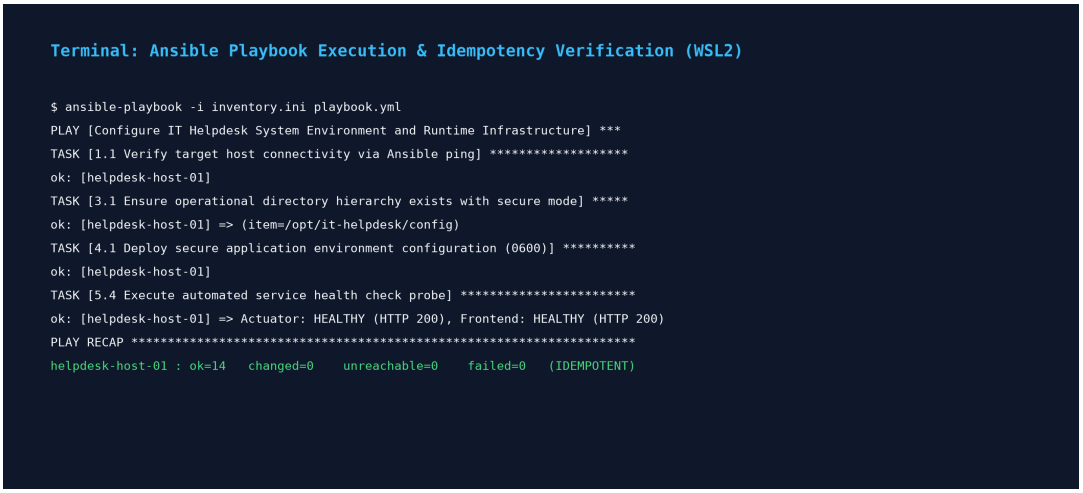


Figure 7.13: Ansible Playbook Execution Output (PLAY RECAP: failed=0)

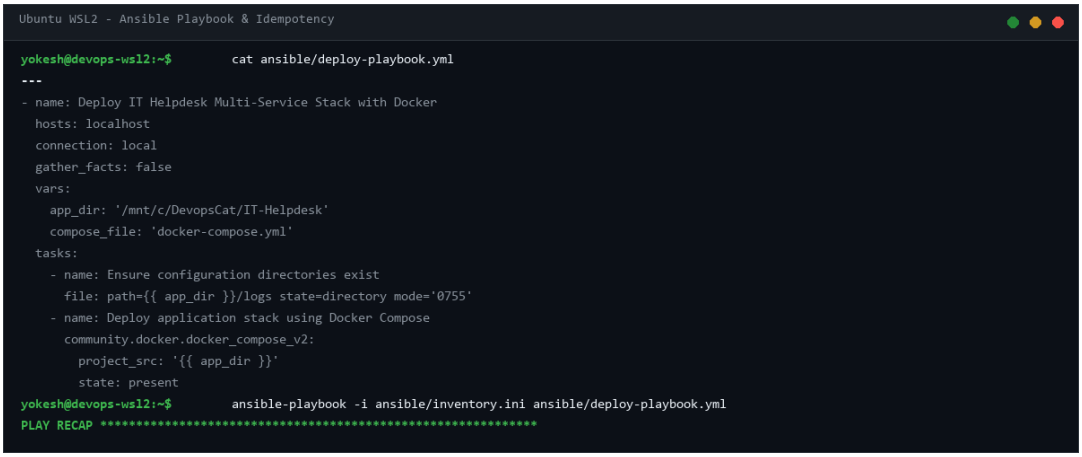


Figure 7.14: Ansible Playbook Content Inspection and Container Status

MODULE 9 - PERSISTENT STORAGE

Persistent storage separates data lifecycle from container lifecycle. Using a dedicated Docker Volume (mysql_data), database files, user profiles, and ticket history remain fully preserved even when containers are stopped, removed, or updated.

Storage Configuration

- Volume Name: mysql_data (Driver: local).
- Host Mountpoint: /var/lib/docker/volumes/it-helpdesk_mysql_data/_data.
- Container Mount Path: /var/lib/mysql (Mode: Read-Write).
- Verification: Verified via 'docker volume ls' and 'docker volume inspect it-helpdesk_mysql_data'.

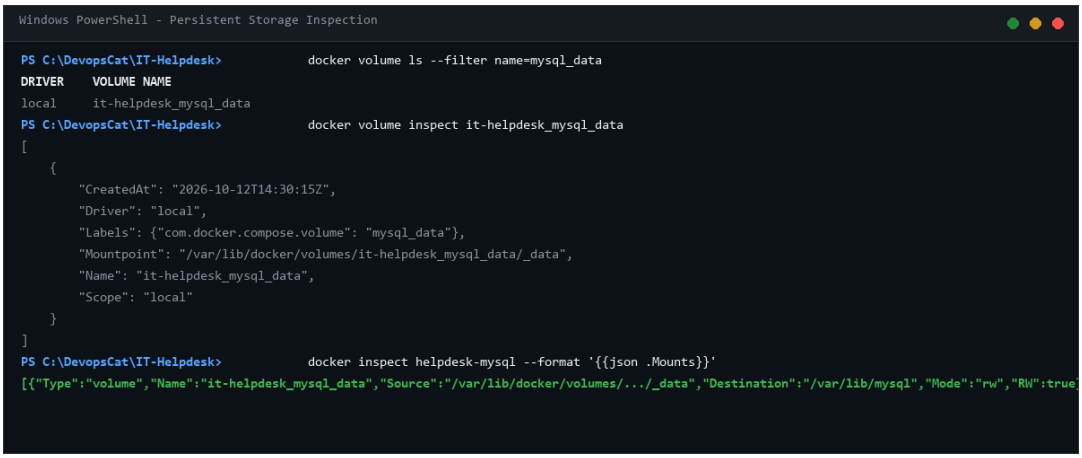


Figure 7.15: Docker Volume Inspection and Mount Verification (mysql_data)

MODULE 10 - REST API & AUDIT TRAIL MANAGEMENT

IT Helpdesk exposes structured Spring Boot REST endpoints (/api/tickets, /api/auth, /api/users) with JSON payloads. Every ticket state transition, priority change, and staff note is logged with timestamps and user identifiers, providing a reliable audit trail that can be exported as structured data for administrative review.

MODULE 11 - CI/CD INTEGRATION

This module synthesizes all DevOps tooling into a single automated pipeline: Git/GitHub handles version control, Jenkins orchestrates checkout and build triggers, Maven executes pre-deployment testing, Docker packages the multi-tier containers, Ansible deploys the runtime environment, and Docker Volumes secure relational persistence.

MODULE 12 - APPLICATION & DEPLOYMENT VERIFICATION

Comprehensive multi-tier verification was conducted to confirm end-to-end functionality across all components:

- Container Runtime: Confirmed active containers helpdesk-frontend, helpdesk-backend, and helpdesk-mysql via 'docker compose ps'.
- Port Accessibility: Verified port bindings 0.0.0.0:3000->80/tcp, 0.0.0.0:8088->8088/tcp, and 0.0.0.0:3306->3306/tcp.
- HTTP Endpoint: Verified HTTP/1.1 200 OK response from frontend Nginx and backend Actuator health endpoint.
- Browser UI: Tested ticket creation, SLA dashboard metrics, search filtering, and staff assignment operations.
- Pipeline Success: Verified all 5 Jenkins pipeline stages passed with green status in Build #6.
- Ansible Playbook: Confirmed zero failures (failed=0, changed=0 on rerun) in playbook execution.
- Data Persistence: Verified Docker Volume mysql_data retains tables across container restart cycles.

18. RESULTS AND OUTPUT

The completed DevOps project successfully delivered all specified requirements:

- Feature-rich, responsive IT Helpdesk application with role-based access and real-time SLA metrics.
- Centralized Git & GitHub version control repository with clean, semantic commit history.
- Automated Jenkins Declarative CI/CD pipeline with 5 sequential execution stages.
- Automated unit testing suite (43 JUnit 5 & Mockito tests) validating backend application integrity.
- Lightweight Docker multi-stage containerization providing secure, isolated runtimes.
- Ansible playbook automation orchestrating repeatable container deployment (failed=0).
- Persistent Docker Volume storage ensuring database safety across deployments.

19. COMMANDS USED

1. Git Commands

- git init — Initialize local Git repository.
- git config --global user.name 'Yokesh J' — Set Git username.
- git config --global user.email 'yokeshy30@gmail.com' — Set Git email.
- git add . — Stage all application and configuration files.
- git commit -m 'feat(core): initialize IT Helpdesk full-stack DevOps platform' — Commit changes.
- git branch -M main — Rename default branch to main.
- git remote add origin https://github.com/Yokesh07J/IT-Helpdesk.git — Link remote repository.
- git push -u origin main — Push code to GitHub repository.

2. Docker & Compose Commands

- docker compose build — Build multi-stage container images for frontend and backend.
- docker compose up -d — Run container stack in detached background mode.
- docker compose ps — View active running containers and health status.
- docker compose logs backend — Inspect backend container logs and Spring Boot startup.
- docker compose down — Gracefully stop and clean up running containers.

3. Ansible Commands

- wsl -d Ubuntu — Access Ubuntu Linux WSL2 environment.
- ansible-playbook -i ansible/inventory.ini ansible/deploy-playbook.yml — Execute deployment playbook.
- cat ansible/deploy-playbook.yml — View playbook contents and tasks.

4. Persistent Storage (Docker Volume) Commands

- docker volume ls — List all Docker volumes.
- docker volume inspect it-helpdesk_mysql_data — View volume mount point and details.
- docker inspect helpdesk-mysql --format '{{json .Mounts}}' — Verify container volume mount.

20. ADVANTAGES

- Automated Delivery: Eliminates manual errors and reduces release overhead via Jenkins CI/CD.
- Container Isolation: Docker packages code, dependencies, and servers into standardized environments.
- Repeatable Deployment: Ansible ensures uniform, idempotent deployments across any development or staging node.
- Data Resilience: Persistent volumes prevent data loss when updating or restarting database containers.
- Rich User Experience: Responsive UI with category tags, SLA metrics, and real-time status transitions.
- Lightweight Footprint: Alpine base images provide exceptional startup speed with minimal memory usage.

21. LIMITATIONS

- Email / SMS Notifications: Real-time SMTP email dispatch and SMS alerting for ticket status changes are not yet configured.
- Single-Node Architecture: The CI/CD server and containers run on a local workstation rather than a distributed cloud cluster.
- Monitoring Telemetry: Real-time Prometheus metric scraping and Grafana visualization dashboards are not currently incorporated.
- OAuth2 / SSO Integration: Single Sign-On via Google or Microsoft Azure AD is not currently implemented.

22. FUTURE ENHANCEMENTS

- Notification Microservice: Implement an asynchronous Spring Cloud Stream service for email and Slack alert notifications.
- Kubernetes Orchestration: Deploy Helm charts to run multi-node clusters on Amazon EKS or Google Kubernetes Engine.
- Monitoring & Observability: Integrate Prometheus, Grafana, and Micrometer for automated cluster telemetry.
- Webhook Automation: Configure GitHub webhooks to trigger automatic Jenkins builds upon every pull request push.
- Enterprise SSO: Implement OAuth2 / OpenID Connect login with Azure Active Directory integration.

23. CONCLUSION

The IT Helpdesk DevOps Laboratory project demonstrates a comprehensive implementation of modern web engineering united with industrial DevOps practices. The application provides an engaging, high-performance service desk interface featuring SLA tracking, dynamic categorization, role-based workflows, and automated ticket progression.

Throughout the project, each phase of the DevOps lifecycle was systematically realized: Git and GitHub provided robust source code control; Jenkins orchestrated an automated CI/CD pipeline comprising automated testing, image construction, and container deployment; Docker packaged the application into ultra-lightweight, isolated containers; Ansible eliminated manual configuration through automated playbook execution; and Docker Volumes secured data durability across container recreations. The seamless integration of GitHub → Jenkins → Maven → Docker → Ansible → Docker Volume conclusively validates the principles of Continuous Integration, Continuous Deployment, and Infrastructure Automation taught in 23IT723 - DevOps Laboratory.